INSON · ИИ-сюрвейер

ТЗ на корректировку после проверки

Ред. 1.0 · 14.10.2026 · основание: проверка коммитов 921c048, d6282bb, e12f148, 9127946 по ТЗ от 13.10.2026 · тестовый сервер ai-surveyor-production.up.railway.app

> **Результат:** этап 1 (2 дня) закрывает критерии приёмки 5, 9 и 10 и исправляет термин в акте. Этап 2 (5 дней) — интерфейс; он начинается только после письменного разрешения трогать app/tg.

1. Правила

- Правила раздела 1 ТЗ от 13.10.2026 действуют: ветка и pull request на каждую задачу, все тесты «плохо 0», только учебные данные, константы в act_settings.
- app/main.py править можно: в нём уже есть изменения этого этапа (заполнение таблиц при старте, маршруты частей продуктов и актуарного отчёта).
- Каждая задача закрывается коммитом, в сообщении которого стоит её номер (К1…К5, И1…И6).

2. Этап 1 — исправления (2 рабочих дня)

К1. Термин «страхование сверх страховой стоимости» вместо «перестрахование»

- В русских текстах о S/V > 1 заменить «перестрахование» на «страхование сверх страховой стоимости». Узбекский и английский тексты уже верные, их не трогать.
- Места: valuation.py — строка 1511 (вывод проверки), SV_TEXT["over"]["ru"] (строка 1877), SV_LABEL["over"] (строка 1881); комментарии в act_engine.py (строка 186) и contract_check.py (строка 8); заголовок раздела Cobalt в отчёте tools/live_check.py.
- Слово «недострахование» в SV_LABEL["under"] заменить на «неполное страхование» (термин ГК РУз, ст. 936).
- Номера статей оставить: ст. 936 и 938 сверены с текстом ГК (library/01_Законодательство/…/ГК РУз глава 52).
- Не трогать тексты про настоящее перестрахование: удержание и MFL в act_texts/scenarios.py, act_texts/rate.py, risk_analytics.py, act_texts/stats.py.

Файлы: `app/valuation.py, app/act_engine.py, app/contract_check.py, tools/live_check.py`

Проверка: grep «перестрах» в valuation.py, contract_check.py и live_check.py — 0 строк. Новый тест: S 150 млн, V 111 млн → ярлык «страхование сверх страховой стоимости», текст содержит «ГК ст. 938»; S 80 млн, V 111 млн → «неполное страхование», «ст. 936».

К2. Поправка на возраст аналога: 0,88 в степени разницы лет

- В ТЗ: $V = \text{median}(P_i \cdot 0{,}95 \cdot 0{,}88^{\Delta} \cdot K_{\text{пробег}} \cdot K_{\text{сост}})$, где Δ = год аналога − год объекта. Сейчас в analog_value 0,88 умножается всегда, без степени — это ошибка.
- Переименовать настройку vehicle.k_market → vehicle.k_age (значение 0,88, границы 0,5–1,0). Для каждого аналога: k = k_age ** (год_аналога − год_объекта). Аналог того же года → k = 1; аналог на 2 года старше объекта → цена делится на $0{,}88^2$ (умножается на 1,29).
- Если у аналога нет года — аналог не участвует в оценке, причина пишется в cut.
- Поиск на avtoelon: сначала год объекта; если после отсева выбросов меньше 3 аналогов — год ± 1, затем ± 2 (с поправкой k_age).
- В act_json.valuation.k и в пояснении к стоимости показывать k_age и Δ для каждого аналога.
- Настройку k_market убрать из act_settings с миграцией: в старой версии настроек ключ игнорируется, а не ломает проверку.

Файлы: `app/valuation.py, app/act_engine.py (VALUE_SETTINGS, проверка границ)`

Проверка: Новый офлайн-тест на снимке 19 аналогов Cobalt 2022 (сохранить в tests/fixtures/avtoelon_cobalt_2022.json): V в диапазоне 105–117 млн сум (ожидается ≈ 108 млн = 95,4 / 0,88). Тест «аналог 2020 для объекта 2022»: $k = 0{,}88^{-2} = 1{,}2913$.

К3. Шрифт PDF на Linux (сервер и тесты)

- app/proposal.py: вместо жёсткого C:\Windows\Fonts\arial.ttf — тот же список кандидатов, что в act_pkg/export.py (FONT_CANDIDATES: Arial для Windows, DejaVu Sans для Linux). Вынести список в одно место (например, app/fonts.py), импортировать в обоих модулях и в exports.py.
- Тесты (test_act.pdf_table и другие места с arial.ttf) берут шрифт из того же app/fonts.py.
- Нет ни одного шрифта — понятная ошибка 503 «На сервере нет шрифта», как сейчас, но в журнале — какие пути проверены.

Файлы: `app/fonts.py (новый), app/proposal.py, app/exports.py, app/act_pkg/export.py, tests/*`

Проверка: В контейнере python:3.12-slim + fonts-dejavu-core (как в Dockerfile): git clone → tools/db_build.py → все 67 наборов «плохо 0», включая test_act, test_proposal, test_registration, test_optimization. На сервере PDF коммерческого предложения отдаётся с кодом 200.

К4. live_check: независимые эталоны и запуск из админки

- В tests/fixtures/live_check_expected.json у каждого случая два блока: snapshot (как сейчас: снимок прошлого прогона, для поиска регрессий) и target (цель из ТЗ, задаётся вручную, --snapshot его не перезаписывает).
- target по случаям: К1 Cobalt — V 105–117 млн и вывод «страхование сверх страховой стоимости» при S 150 млн; К5 и ТЗ магазин 450 м² — V = формула затратного метода по build_cost_m2 (±1 сум, считается в тесте); К6 склад — ставка ≥ 0,05%; Р1–Р5 — премия = сумма частей (±1 сум) и ставка каждой части ≥ её минимума; D1–D3 — договор прочитан, ≥ 8 из 10 полей, 12 проверок есть; у всех — PDF ≤ 2 страниц и ≤ 60 с.
- Итог отчёта — две строки: «цели ТЗ: N из 20», «без регрессий: N из 20». Приёмка — по первой.
- У каждого случая вне допуска в отчёте — что именно не сошлось (поле, ожидание, факт).
- Если avtoelon недоступен с сервера, К1 получает статус «нет доступа к источнику», а не «вне допуска»; цель К1 при этом закрывается офлайн-тестом К2.
- Маршрут POST /admin/live-check (только администратор): запускает run(url, telegram=True) в фоновом потоке, один запуск одновременно; GET /admin/live-check/last — последний отчёт.

Файлы: `tools/live_check.py, tests/fixtures/live_check_expected.json, app/main.py` или `app/admin*`

Проверка: Запуск из админки на тестовом сервере: отчёт приходит в Telegram и лежит в data/reports. «Цели ТЗ: 20 из 20» (К1 — «нет доступа», если сервер вне Узбекистана).

К5. Отчёт о сдаче этапа 1

- Приложить отчёт live_check с сервера (md) и вывод прогона тестов в контейнере (К3).
- Для двух случаев, не прошедших в прошлом прогоне (18 из 20), — какие это были случаи и чем закрыты.

Проверка: Оба файла приложены к pull request этапа 1.

Приёмка этапа 1

№	Критерий ТЗ 13.10.2026	Чем закрывается
5	Cobalt ≈ 111 млн, верный вывод по S/V	К1 + К2, офлайн-тест 105–117 млн
9	20 из 20 осмотров в допуске	К4: «цели ТЗ: 20 из 20»
10	Тесты на чистой машине	К3: 67 наборов на Linux

3. Этап 2 — интерфейс (5 рабочих дней, после разрешения)

Сервер под эти задачи готов. Формулировки, проверки и макет — из ТЗ от 13.10.2026 (задачи 1.3, 2.2, 5.2, 7.1, 8.1, 9.1 и приложение А).

№	Задача	Проверка
И1 (1.3)	Марка, модель и год из полей класса сами заполняют «Что ищем»	Chevrolet Cobalt 2022 — поле заполнено без клика
И2 (2.2)	Вкладка «Части» в карточке продукта (просмотр, правка, новая версия)	0833 из 2 частей правится в админке, версия +1
И3 (5.2)	Кнопки «Принять рыночную» и «Оставить с пояснением» на экране «Стоимость»	После «Принять» S = V и премия пересчитана; пояснение попадает в акт

№	Задача	Проверка
И4 (7.1)	Условия из договора подставляются в поля осмотра с пометкой «из договора»	Учебный договор DOCX: ≥ 8 полей заполнены, пометка видна
И5 (8.1)	Пять экранов: Фото → Объект → Стоимость → Цена → Акт	Осмотр Cobalt проходит за ≤ 5 экранов, без дублей полей
И6 (9.1)	Экран «Акт» показывает то же, что PDF	Числа экрана и PDF совпадают на 20 осмотрах live_check

Если разрешения на интерфейс нет, этап 2 не начинается. Критерий 6 (0833 через админку) тогда остаётся частично выполненным.